

Module 4: Supervised Learning

Submodule: 4.1 Linear Regression

Lecture: Gradient Descent

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I	Introduction	1
II	Gradient Descent Overview	1
III	Mini-Batch Gradient Descent	3
	Updated Gradient Calculation	4
	Trade-offs in Mini-Batch Gradient Descent	4
IV	Summary	5

I. Introduction

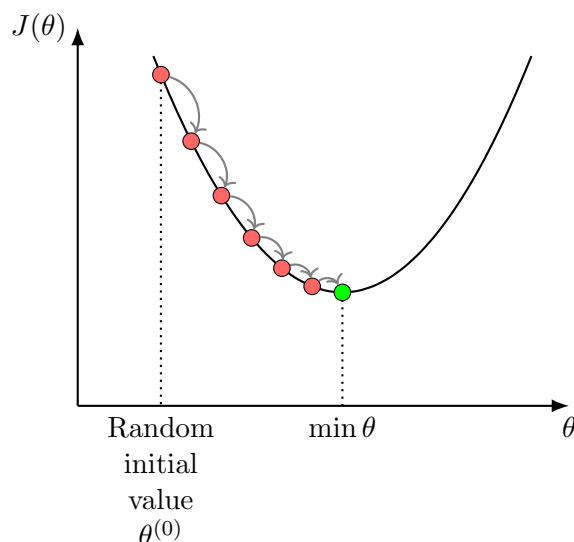
The goal of this discussion is to understand how to find the parameters θ using a method called **gradient descent**. Gradient descent is an iterative optimization algorithm used to minimize a function, particularly in the context of machine learning.

II. Gradient Descent Overview

To find the optimal value of θ that minimizes the function $J(\theta)$, we employ an iterative search algorithm. This algorithm begins with an initial guess for θ and continuously updates it to reduce $J(\theta)$. The process continues until θ converges to a value that minimizes $J(\theta)$. One widely used approach for this optimization is the **gradient descent** algorithm.

Assume we have a cost function $J(\theta)$ that we wish to minimize. Imagine that it appears as illustrated below. (Its important to note that this is a generic representation that may not accurately reflect the specific $J(\theta)$ we

discussed earlier. Our goal here is to understand how the gradient descent algorithm operates in relation to this hypothetical loss function.)



If $J(\theta)$ is convex (bowl-shaped), any local minimum found during the gradient descent process will also be a global minimum. This characteristic is particularly useful in least squares regression since the cost function is convex, which ensures that the solution is optimal.

The process of gradient descent involves the following steps:

1. Initialization - Begin at a random or predefined point (e.g., $\theta^{(0)} := 0$).
2. Iterative Updates - For each iteration t (timestep):

$$\theta_j^{(t+1)} := \theta_j^{(t)} - \alpha \frac{\partial}{\partial \theta_j} J(\theta^{(t)}), \quad j = 0 \dots d.$$

α is a positive hyperparameter known as the **learning rate** or **step size**. It adjusts how quickly we traverse the cost curve. There are theories on how to set α which we'll cover later. We will compute the partial derivative for each "direction" j . ($:=$ means "assignment".)

The derivative is also called the **gradient** of $J(\theta)$ at $\theta^{(t)}$ which indicates the direction of steepest *ascent*. (We'll be subtracting the gradient in order to *descend* the cost curve.)

$$\text{Gradient} := \frac{\partial}{\partial \theta_j} J(\theta^{(t)}) = \nabla J(\theta^{(t)})$$

In order to implement this algorithm, we have to work out the partial derivative term on the right hand side. We have:

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta) &= \sum_{i=0}^n \frac{\partial}{\partial \theta_j} \frac{1}{2} \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right)^2 && \text{by definition} \\
&= 2 \cdot \frac{1}{2} \sum_{i=0}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_j} \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right) \\
&= \sum_{i=0}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{m=0}^d \theta_m x_m^{(i)} - y^{(i)} \right) \\
&= \sum_{i=0}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right) x_j^{(i)}
\end{aligned}$$

Recall that $h_{\theta}(x^{(i)}) = \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} \dots \theta_j x_j^{(i)} = \sum_{j=0}^d \theta_j x_j^{(i)}$. Thus,

$$\theta_j^{(t+1)} := \theta_j^{(t)} - \alpha \sum_{i=0}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right) x_j^{(i)}, \quad j = 0 \dots d \quad (1)$$

The rule is called the **LMS** update rule (LMS stands for "least mean squares"), and is also known as the **Widrow-Hoff** learning rule.

By grouping the updates of the coordinates into an update of the vector θ , We can rewrite update (1) in a slightly more succinct way as:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha \sum_{i=0}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right) x_j^{(i)}, \quad j = 0 \dots d \quad (2)$$

In conclusion, gradient descent is a powerful optimization technique that extends beyond least squares regression, making it adaptable for various machine learning problems. By incrementally updating parameters in the direction opposite the gradient, we can efficiently converge to a minimum of the cost function.

III. Mini-Batch Gradient Descent

In machine learning, we often deal with high volumes of data. When using standard **batch gradient descent**, calculations involve utilizing the entire dataset, which can be time-consuming and inefficient, especially with very large datasets. This method requires examining every single data point to compute a gradient for updating the model, thus making it slow.

To address this issue, the concept of **mini-batch gradient descent** was introduced. This approach randomly selects a subset (mini-batch) of

data points, significantly speeding up the computation while allowing for effective gradient estimation. A mini-batch consists of a small number of points. For example, from a dataset with one million data points, we might select B to be as small as 16 or even just 1. Often B is chosen to optimize training performance based on the amount of GPU memory.

$$B = \{i_1, i_2, \dots, i_b\}, \quad b < n. \quad (\text{Training set})$$

When $B = 1$, meaning we update the parameters with a single training example, the algorithm is referred to as **stochastic gradient descent (SGD)**. There are “intelligent” methods for selecting the examples to be included in a batch which improves performance over selecting randomly.

Updated Gradient Calculation

Instead of calculating the gradient over the entire dataset as in (2), we sum the contributions from the selected mini-batch. The gradient *estimation* can be represented as:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha_b \sum_{k \in B} \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right) x_j^{(i)}.$$

Note:

- The learning rate used in mini-batch updates α_b might differ from that used in full batch updates α . There is an important connection between the batch size and the learning rate for achieving effective convergence.
- It’s beneficial to ensure that each point in the dataset has a chance of being included in some mini-batch, often achieved by shuffling the data.

Trade-offs in Mini-Batch Gradient Descent

Mini-batch gradient descent offers several trade-offs between speed and accuracy:

- Mini-batching tends to converge in terms of steps to reach a solution faster than batch gradient descent because B is significantly smaller than n , leading to fewer computations. However, one downside is that each estimate based on fewer data points may produce noisier gradients, raising concerns about convergence accuracy.
- In practice, many machine learning practitioners utilize mini-batch gradient descent as it assists in managing redundancy in data effectively. Smaller mini-batch sizes can enhance convergence speed, even if they introduce some noise into the model.

Overall, mini-batch gradient descent represents a significant advancement over the traditional batch method, striking a balance between computational efficiency and performance. It has become a standard approach for training large models in the field of machine learning.

IV. Summary

- **Gradient Descent Purpose:** Gradient descent is an iterative optimization algorithm used to find the optimal parameter θ that minimizes a cost function $J(\theta)$. It is fundamental for training machine learning models.
- **Initialization and Iteration:** The algorithm starts with an initial guess for θ and continuously updates it using the derivative (gradient) of the cost function until convergence is reached.
- **Convergence to Local Minima:** With convex cost functions, any local minimum found during iteration will also be a global minimum, ensuring an optimal solution in least squares regression.
- **LMS Update Rule:** The update rule can be succinctly expressed using the Least Mean Squares (LMS) update formula, which incorporates the learning rate α and derivatives of the cost function.
- **Mini-Batch Gradient Descent:** The introduction of mini-batch gradient descent allows for the efficient processing of datasets by only using a random subset (mini-batch) for gradient calculations, improving speed and computational efficiency.
- **Trade-offs of Mini-Batching:** While mini-batching leads to faster convergence, it may also introduce noise due to fewer samples being used to estimate gradients. Practitioners must balance batch size and learning rate for effective model training.
- **Adaptability of Gradient Descent:** Gradient descent is a versatile optimization technique that applies to various machine learning problems, extending beyond just linear regression.